Express JS

Express JS

- Express.js is a **fast, flexible and minimalist web framework** for Node.js.
- It’s effectively a tool that **simplifies building web applications** and APIs using JavaScript on the server side.
- Express is an **open-source** that is developed and maintained by the Node.js foundation.
- Express.js offers a robust set of features that enhance your productivity and streamline your web application.
- It makes it easier to organize your application’s functionality with **middleware and routing**.
- It adds helpful utilities to Node HTTP objects and facilitates the rendering of dynamic HTTP objects.

Express JS

- Express is a **user-friendly framework** that simplifies the development process of Node applications.
- It uses JavaScript as a programming language and provides an efficient way to build web applications and APIs.
- With Express, you can **easily handle routes, requests, and responses**, which makes the process of creating robust and scalable applications much easier.
- Moreover, it is a **lightweight and flexible framework** that is easy to learn and comes loaded with middleware options.
- Whether you are a beginner or an experienced developer, Express is a great choice for building your application.

Key Features of Express

- **Middleware and Routing**: Define clear pathways (routes) within your application **to handle incoming HTTP requests** (GET, POST, PUT, DELETE) with ease. Implement **reusable functions** (middleware) to **intercept requests and create responses**, adding functionalities like authentication, logging, and data parsing.

- **Minimalistic Design**: Express.js follows a simple and minimalistic design philosophy. This simplicity allows you to **quickly set up a server, define routes, and handle HTTP requests efficiently**. It's an excellent choice for building web applications without unnecessary complexity.

- **Templating Power**: Incorporate templating engines like Jade or EJS to **generate dynamic HTML** content, **enhancing user experience.**

Key Features of Express

- **Flexibility and Customization**: Express.js **doesn’t impose a strict application** architecture. You can structure your code according to your preferences. Whether you’re building a RESTful API or a full-fledged web app, **Express.js adapts to your needs**.

- **Static File Serving**: Effortlessly **serve static files** like images, CSS, and JavaScript from a designated directory within your application.

- **Node.js Integration**: Express.js **seamlessly integrates with the core functionalities of Node.js**, allowing you to harness the power of asynchronous programming and event-driven architecture.

Installation

npm install express

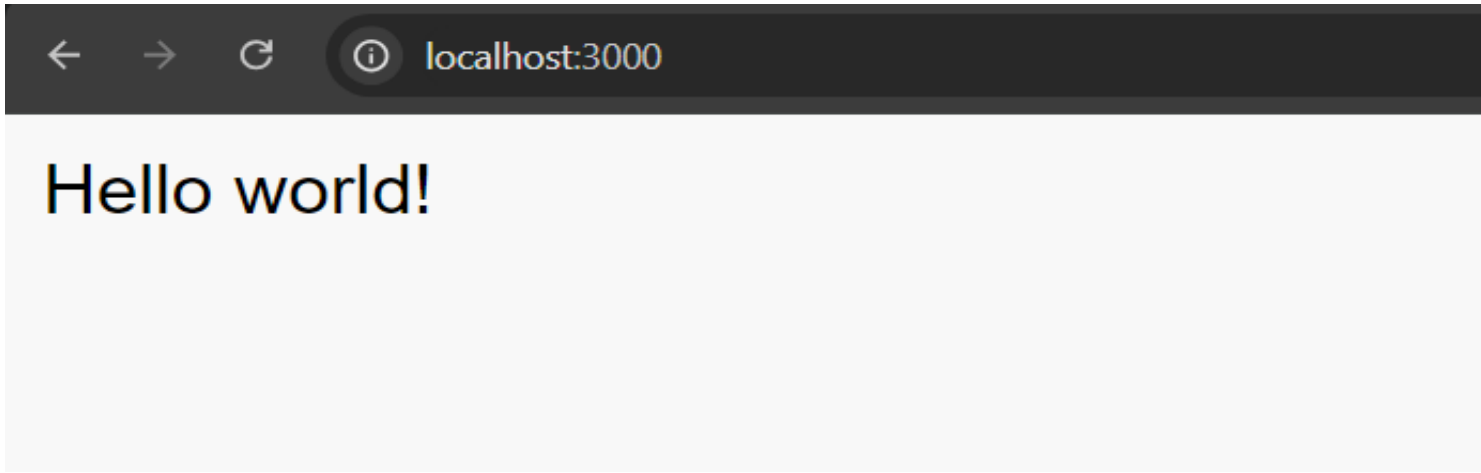
Example - exp.js

on terminal: node exp.js

```
var express = require('express');
var app = express();


app.get('/', function(req, res){
   res.send("Hello world!");
});
app.listen(3000);
```

Browser: http://localhost:3000/



Express Example

- **require('express')** is a Node.js function that **loads the Express library** so that you can use it in your code.
- **express() function** is called to **create a new Express application**, which is essentially an object that represents the web application. This object allows you to **define routes, middleware, and other behavior for your web server**.
- **app.get('/', function(req, res)**: This line defines a **GET route at the path /** (the root URL of the server).
- **app.listen(3000);**This **starts the web server** and **listens for incoming requests on port 3000**.

Request Object

- Express.js Request and Response objects are the parameters of the callback function which is used in Express applications.

- The express.js request object represents the **HTTP request** and has properties for the request **query string, parameters, body, HTTP headers**, and so on.

Request Object

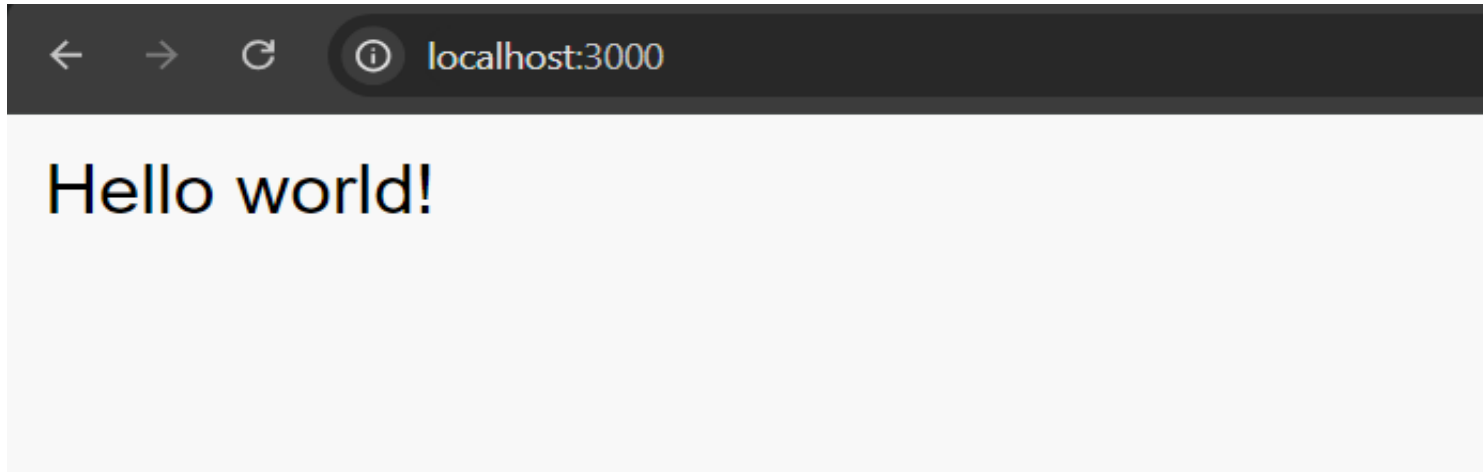
Index	Properties	Description
1.	req.app	This is used to hold a **reference to the instance** of the express application that is using the middleware.
2.	req.baseurl	It specifies the **URL path** on which a router instance was mounted.
3.	req.body	It contains key-value pairs of data submitted in the request body. By default, it is undefined, and is populated when you use body-parsing middleware such as body-parser.
4.	req.cookies	When we use cookie-parser middleware, this property is an object that contains cookies sent by the request.
5.	req.fresh	It specifies that the request is "fresh." it is the opposite of req.stale.
6.	req.hostname	It contains the hostname from the "host" http header.

Example - exp1.js

on terminal: node exp1.js

```
var express = require('express');
var app = express();
app.get('/', function(req, res){
    console.log(req.app);
   res.send("Hello world!");
});
app.listen(3000);
```

Browser: http://localhost:3000/



Response Object

- The Response object (res) specifies the **HTTP response** which is sent by an Express app when it gets an HTTP request.
- It sends **response back to the client browser**.

Response Object

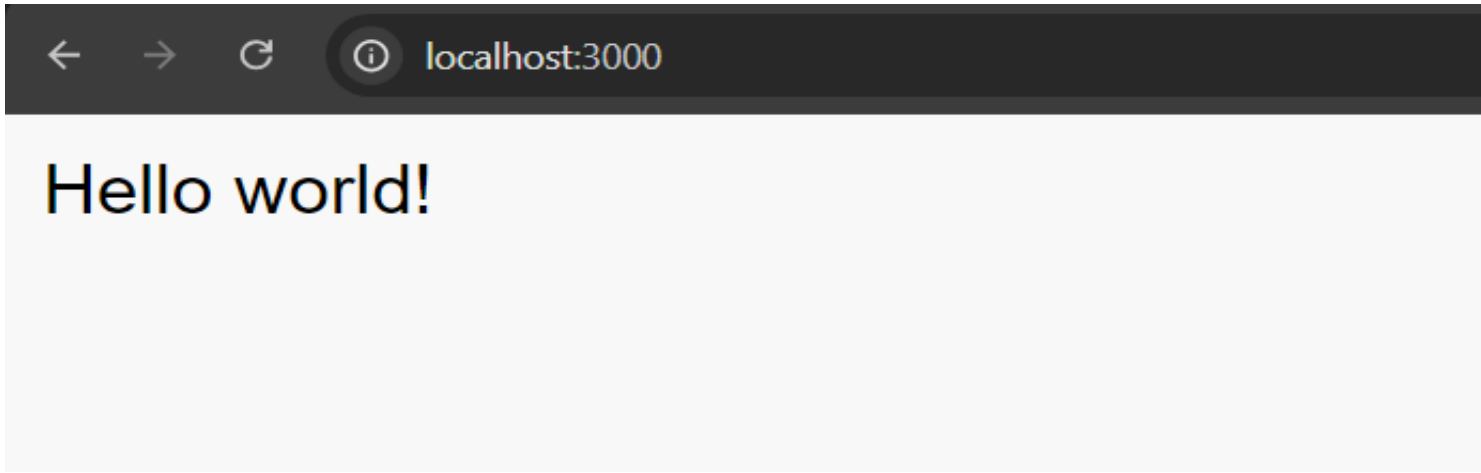
Index	Properties	Description
1.	res.app	It holds a reference to the instance of the express application that is using the middleware.
2.	res.headersSent	It is a Boolean property that indicates if the app **sent HTTP headers for the response**.
3.	res.locals	It specifies an object that contains **response local variables** scoped to the request

Example - exp2.js

on terminal: node exp2.js

```
var express = require('express');
var app = express();
app.get('/', function(req, res){
    console.log(res.app);
   res.send("Hello world!");
});
app.listen(3000);
```

Browser: http://localhost:3000/



Routing in Express

- Routing is made from the **word route**.
- It is used to determine the **specific behavior of an application**.
- It specifies how an application responds to a **client request to a particular route**, URI or path and a specific HTTP request method (GET, POST, DELETE and PUT).
- It can handle different types of HTTP requests.

HTTP Method

- The HTTP method is supplied in the request and specifies the operation that the client has requested.

- The following table lists the most used HTTP methods −

S.No.	Method & Description
1	**GET:** The GET method requests a representation of the specified resource. Requests using GET should only retrieve data and should have no other effect. The GET method **send data through URL**
2	**POST:** The POST method requests that the server accept the data enclosed in the request as a new object/entity of the resource identified by the URI. The POST method send large amount of data because data is sent in the body.
3	**PUT:** The PUT method requests that the server accept the data enclosed in the request as a modification to existing object identified by the URI. If it **does not exist** then the PUT method should **create one**.
4	**DELETE:** The DELETE method **requests that the server delete the specified resource**.

GET – HTTP method

- Get method facilitates you to **send only limited amount of data** because data is sent in the header.
- It is not secure because **data is visible in URL bar**.

POST – HTTP method

- Post method facilitates you to **send large amount of data because data is send in the body.**
- Post method is secure because data is **not visible in URL** bar but it is not used as popularly as GET method.
- On the other hand GET method is more efficient and used more than POST.

Example - exp3.js

```
var express = require('express');

var app = express();

app.get('/', function(req, res){

 res.send("Hello world!");

});

app.get('/hi', function(req, res){

    res.send("Hi!");

});

app.get('/welcome', function(req, res){

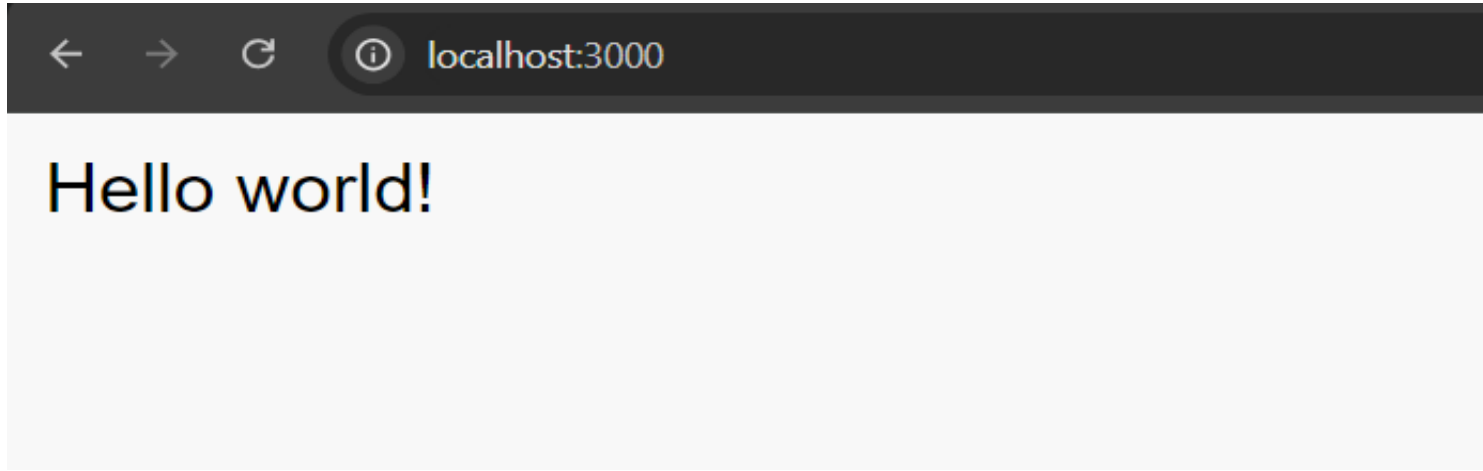
    res.send("Welcome");

});

app.listen(3000);
```

on terminal: node exp3.js

Browser: http://localhost:3000/
http://localhost:3000/hi
http://localhost:3000/welcome



all – HTTP method

- A special method, **all**, is provided by Express to **handle all types** of http methods at a particular route using the same function.

Example - exp4.js

```
var express = require('express');

var app = express();

app.all('/', function(req, res){

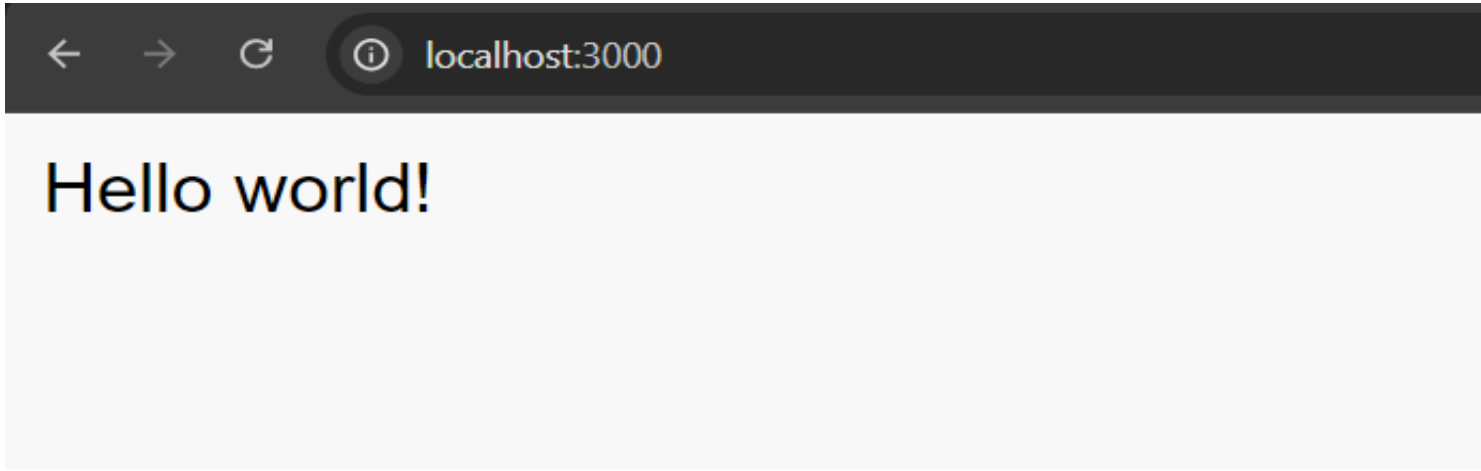
 res.send("Hello world!");

});

app.listen(3000);
```

on terminal: node exp4.js

Browser: http://localhost:3000/



use – HTTP method

- The router.use() function uses the **specified middleware function or functions**.
- It basically mounts middleware for the **routes** which are being served by the specific router.

Example - exp5.js

```
var express = require('express');

var app = express();

app.use('/', function(req, res){

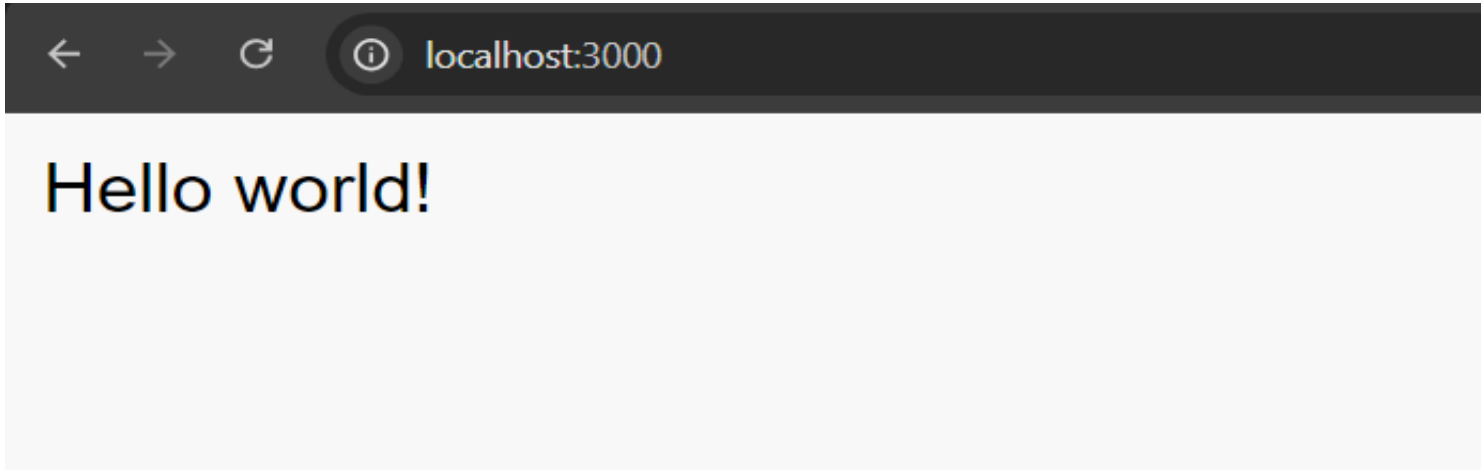
 res.send("Hello world!");

});

app.listen(3000);
```

on terminal: node exp5.js

Browser: http://localhost:3000/



Express JS Template

- A template engine facilitates you to **use static template files** in your applications.
- At runtime, **it replaces variables in a template file** with **actual values**, and transforms the **template into an HTML file** sent to the client.
- So this approach is preferred to **design HTML pages** easily.

Popular template engines that work with Express.js

- **Pug**
- mustache
- dust
- atpl
- eco
- ect
- ejs
- haml
- haml-coffee
- handlebars
- hogan
- jazz
- jqtpl
- JUST
- liquor
- QEJS
- swig
- templayed
- toffee
- underscore
- walrus
- whiskers

Pug Template

- Pug is a popular **template engine** for Express.js applications, used to **generate HTML dynamically with a simplified, indentation-based** syntax
- Pug is a **template engine** for Node.js and Express that allows developers to write HTML templates in a cleaner, more concise way.
- Pug files use **indentation to structure HTML**, and JavaScript code can be embedded within the **template to dynamically generate HTML**.

Key Features

- **Simplified Syntax**: Eliminates closing tags and uses indentation, making **HTML easier to read and maintain**.
- **Dynamic Content Rendering**: Supports embedding JavaScript, allowing for variables, loops, and conditionals
- **Template Inheritance:** Allows for layout files, partials, and reusability with includes and mixins
- **Fast Rendering:** Compiles to JavaScript functions, so it’s fast and efficient for server-side rendering.

Installations

npm install express pug

Pug Template

- Template engine makes you able to **use static template files in your application**.
- To render template files you have to set the following application setting properties:

app.set('views', './views')

- Views: It **specifies a directory where the template files are located**.
- view engine: It specifies the **template engine** that you use.
- For example, to use the Pug template engine:

app.set('view engine', 'pug').

Example - index.pug

```
doctype html
html
  head
    title A simple pug template example
  body
    h1 This page is produced by pug template engine
    p some paragraph here..
```

Create the “**views**” folder in the location where server.js is stored

Store the index.pug inside the views

Example - server.js

```
var express = require('express');
var app = express();
//set view engine
app.set("view engine","pug")
app.get('/', function (req, res) {
res.render('index');
});
var server = app.listen(5000, function () {
    console.log('Node server is running..');
});
```

on terminal: node server.js

Browser: http://localhost:5000/

Example - output

